

Пять идей школьного проекта на основе классификатора 3D-моделей

Как превратить обученную на ModelNet40 нейросеть и конвертер форматов в проект, решающий реальную задачу

12 сентября 2026

Что уже есть и чего не хватает

На данный момент готовы два компонента:

1. **Классификатор** — нейросеть, обученная на ModelNet40 (40 классов бытовых объектов: chair, table, sofa, bed, lamp, vase, bottle, cup, airplane, car, ...). Она принимает модель в формате OFF и возвращает класс и вероятности.
2. **Конвертер** — скрипт, который переводит любой распространённый формат (STL, OBJ, PLY, GLB/GLTF, FBX, 3MF, STEP...) в OFF.

Вместе это «движок»: *любой 3D-файл* → *метка класса*. Сам по себе движок никому не нужен; проектом он становится, когда встроен в чью-то рабочую задачу. Ниже — пять таких задач. Все они оставляют классификатор центральной частью системы, а доработка сводится к небольшому числу скриптов вокруг него (рис. 1).

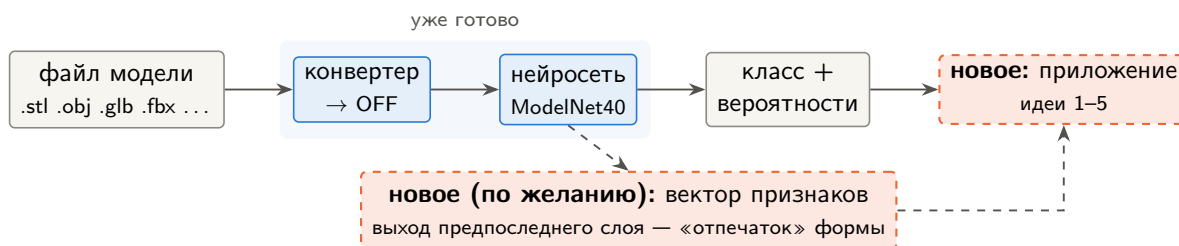


Рис. 1: Общая схема. Синее — готовые компоненты, оранжевое пунктиром — то, что добавляется в проекте. Вектор признаков (эмбединг) нужен идеям 1 и 3: это одна строка кода, которая берёт выход слоя перед финальным softmax.

Об оценках трудоёмкости. Оценки даны для школьника, у которого классификатор и конвертер уже работают, в расчёте на 3–6 часов работы в неделю. Звёздочки: ★ — пара скриптов на выходные, ★★ — 1–2 недели, ★★★ — 3–4 недели, ★★★★ — 1–2 месяца с обучением новой модели.

Идея 1. Умный органайзер библиотеки 3D-моделей

Сложность: ★★ **Новое обучение:** не требуется **Тип результата:** консольная утилита + HTML-каталог

Проблема. У любого, кто печатает на 3D-принтере или делает игры, копится папка из сотен файлов с именами вроде `final_v3.stl`, `Untitled.obj`, `model(2).stl`. В школьном кружке 3D-печати эта папка общая, и найти в ней «ту подставку под телефон» невозможно. Люди скачивают модель заново, потому что не могут найти уже скачанную.

Идея. Скрипт, который проходит по папке, конвертирует каждый файл в OFF, классифицирует и раскладывает модели по подпапкам-категориям (`chair/`, `vase/`, `lamp/...`), дописывая класс в имя файла. Дополнительно строит HTML-страницу-каталог с миниатюрой каждой модели, классом и уверенностью, а также умеет искать «похожие модели» по вектору признаков сети (рис. 2).

Цель. Библиотека из 500 разрозненных файлов за одну команду превращается в структурированный каталог с поиском, без ручной разметки.

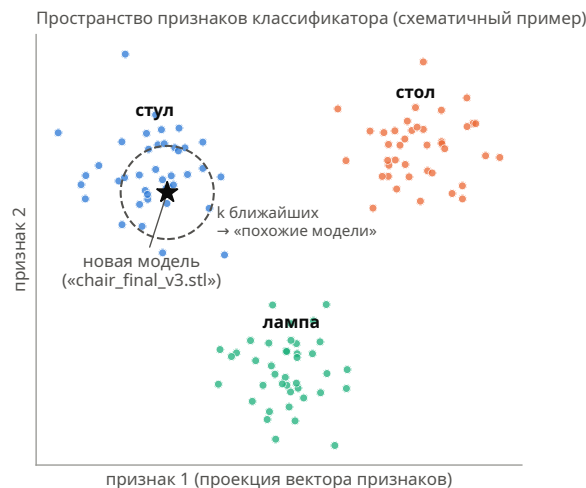


Рис. 2: Вектор признаков (эмбединг) как ключ поиска. Модели одного класса лежат рядом; ближайшие соседи новой модели — это «похожие модели», в том числе дубликаты под другими именами. Данные на рисунке схематичные.

Этапы работы

1. **Пакетный прогон** (`organize.py`). Обход папки, вызов конвертера и классификатора для каждого файла, сохранение результата в `catalog.csv`: путь, класс, уверенность, число вершин, габариты. Обработанные файлы запоминать по хэшу, чтобы не считать повторно.
2. **Раскладка и переименование**. Копирование (не перемещение, для безопасности) в `sorted/<класс>/`; при уверенности ниже порога (например 0,5) — в папку `unsure/`, чтобы человек проверил вручную.
3. **Миниатюры**. Рендер каждой модели в PNG с одного ракурса (`matplotlib` или `trimesh` + `pyrender`, 200×200 px).
4. **HTML-каталог**. Одна страница с сеткой миниатюр, фильтром по классу и поиском по имени (шаблон на Jinja2 или простая f-строка). Открывается в браузере без сервера.
5. **Поиск похожих** (усложнение). Сохранить эмбединг каждой модели, для запроса считать косинусное расстояние и выводить 5 ближайших. Это же находит дубликаты: расстояние почти ноль.
6. **Проверка на реальной папке**. Взять 100–200 моделей с Thingiverse/Printables, посчитать, какую долю скрипт разложил верно, и честно показать, где ошибается (например, модели вне 40 классов).

Что показать на защите. Терминал: одна команда — и папка «до» превращается в папку «после»; каталог в браузере; демонстрация «найди похожие»: загружаем модель стула, получаем 5 стульев. Слабое место, о котором лучше сказать самому: объекты вне 40 классов ModelNet попадают в ближайший похожий класс, поэтому порог уверенности и папка `unsure/` обязательны.

Идея 2. Помощник 3D-печати: подбор профиля слайсера по типу объекта

Сложность: ★★ (★★★ с собственным датасетом) **Новое обучение:** по желанию **Тип результата:** плагин-скрипт перед слайсером

Проблема. Новичок в кружке 3D-печати открывает слайсер (Cura, PrusaSlicer) и видит десятки настроек: заполнение, поддержки, ориентация, толщина стенок. Ваза, напечатанная с заполнением 20 % и поддержками, тратит вдвое больше пластика и времени, чем в режиме «спиральная ваза»; фигурка, положенная набок, обрастает поддержками, которые портят поверхность. Опытный человек определяет нужный режим по виду объекта за секунду — именно это и может делать классификатор.

Идея. Скрипт получает модель, определяет класс и по таблице «класс → профиль печати» выдаёт рекомендации: режим (ваза/обычный/прочный), заполнение, поддержки, ориентация на столе, и сохраняет готовый профиль слайсера (.curaprofile или .ini PrusaSlicer). Ориентация подбирается простым перебором: модель поворачивают 6–24 способами и считают площадь нависающих граней (рис. 3).

Цель. Сократить число неудачных печатей и расход пластика у начинающих: нажал кнопку — получил разумные настройки и объяснение, почему они такие.

Класс «vase» → дном вниз, без поддержек

Та же модель набор → нужны поддержки

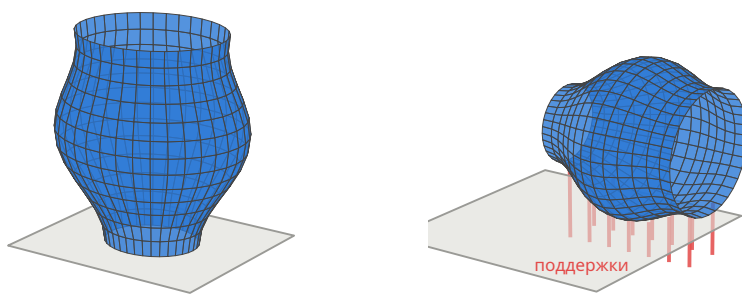


Рис. 3: Класс подсказывает режим (для вазы — «спираль», без заполнения), а геометрия — ориентацию: нависающие грани с нормалью, направленной вниз под углом больше 45°, требуют поддержек; их площадь минимизируют перебором поворотов.

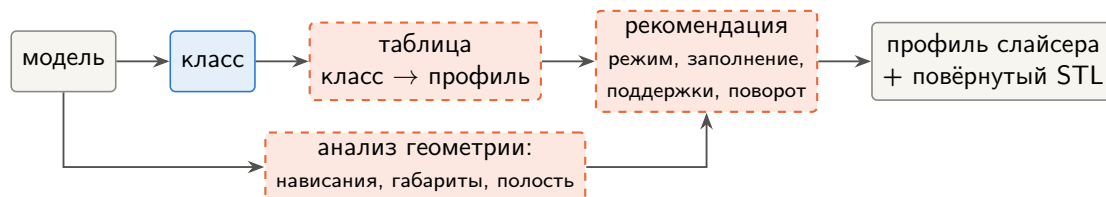


Рис. 4: Схема помощника печати. Классификатор отвечает «что это», простая геометрия — «как это лучше положить».

Этапы работы

1. **Таблица правил.** Для классов ModelNet40, которые реально печатают (vase, cup, bottle, bowl, lamp, plant, flower_pot, chair, table, airplane, car, guitar, person...), записать в YAML профиль: режим, заполнение, стенки, поддержки, рекомендуемая ориентация. Источник — советы опытных печатников и собственные тесты.
2. **Анализ геометрии** (geometry.py). По OFF-сетке считать: габариты в мм, площадь опоры на стол, долю граней с нормалью вниз (нависания), наличие закрытой полости (для режима

«ваза»).

3. **Подбор ориентации.** Перебор поворотов на 90° вокруг трёх осей (24 варианта), для каждого — площадь нависаний и площадь опоры; выбрать лучший и сохранить повёрнутый STL.
4. **Экспорт профиля.** Сгенерировать файл настроек Cura или PrusaSlicer из шаблона, приложить текстовое объяснение («это ваза, поэтому спиральный режим и 0 % заполнения»).
5. **Проверка.** Напечатать 5–6 объектов с рекомендованными и с настройками «по умолчанию», сравнить расход пластика, время и качество — это и будет главный результат в презентации.
6. **Усложнение: свой датасет.** ModelNet40 не знает типичных «кружковых» объектов: брелок, подставка под телефон, шестерня, коробочка с крышкой. Добавить 4–6 таких классов по 60–100 моделей (Thingiverse + собственные модели в Tinkercad с вариациями) и дообучить сеть на ModelNet10 + новые классы.

Зачем здесь классификатор: честная оговорка

Нависания, ориентация и толщина стенок считаются из геометрии, и класс для этого не нужен. Если бы проект состоял только из этого, нейросеть была бы пришта для галочки, и на защите об этом спросят. Класс добавляет то, чего в геометрии нет: *назначение* предмета и его *ожидаемый размер*.

- **Назначение и режим.** Полый предмет с открытым верхом может быть вазой, стаканом, абажуром или миниатюрой ведра. Для вазы нужен спиральный режим с одной стенкой, для стакана герметичные стенки в 3–4 периметра, для абажура тонкие просвечивающие стенки, для миниатюры мелкий слой и качество поверхности. Геометрически это один и тот же объект.
- **Прочность против красоты.** Кронштейн, шестерня или крючок требуют заполнения 40–60 % и больше периметров; фигурка или ваза печатаются с 5–10 %. Профиль выбирают по назначению, а не по форме: именно это опытный печатник делает «на глаз».
- **Масштаб.** В STL нет единиц измерения, путаница мм/дюймы/см встречается постоянно. Если сеть говорит «стул», а модель 40 мм высотой, это либо миниатюра, либо ошибка единиц. Класс даёт ожидаемый реальный размер, и скрипт может предупредить: «похоже, модель в дюймах, увеличить в 25,4 раза?». Без класса такую проверку сделать нельзя.
- **Семантический верх.** Перебор поворотов по минимуму поддержек может поставить фигурку на голову или вазу горлышком вниз. Класс подсказывает, где у предмета низ и какие поверхности должны остаться чистыми.

Переформулировка. В работе явно разделить две части. Геометрический анализ отвечает на вопрос «как положить и где поддержки» и не требует нейросети. Классификатор отвечает на вопрос «что это и зачем» и переводит класс в *функциональные требования*: герметичность, прочность, точность, качество поверхности, ожидаемый размер. Профиль слайсера собирается из обеих частей (рис. 5).

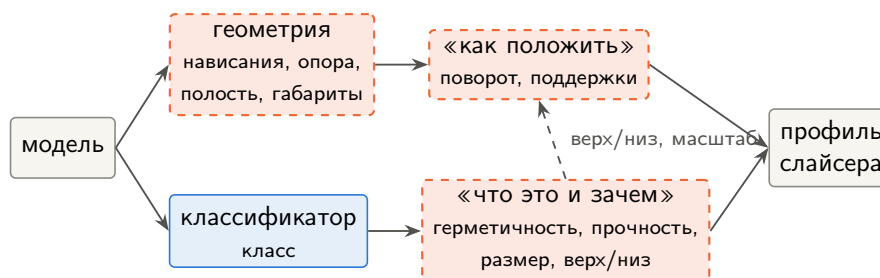


Рис. 5: Честное разделение ролей. Без нейросети работает только верхняя ветка; нижняя ветка добавляет назначение и размер, которых в геометрии нет.

Контрольный эксперимент. Напечатать одни и те же 5–6 объектов с тремя наборами настроек: «по умолчанию», «только геометрия» и «геометрия + класс», и сравнить расход пластика, время и результат (герметичность вазы, прочность крючка, качество фигурки). Если класс не улучшает результат, это тоже честный вывод, и он тоже годится для защиты.

Оценка. Даже в такой формулировке вклад классификатора в идею 2 меньше, чем в остальных четырёх: там без класса задача не решается вовсе, здесь он лишь улучшает решение. Идею 2 стоит брать при наличии принтера и желании получить физический результат; иначе разумнее идеи 3 или 5, а геометрический подбор ориентации добавить туда как небольшой модуль.

Идея 3. Автопроверка учебных заданий по 3D-моделированию

Сложность: ★★ **Новое обучение:** не требуется **Тип результата:** Telegram-бот или веб-страница на Flask/Streamlit

Проблема. На уроках информатики и технологии ученики моделируют в Tinkercad, Blender, SketchUp, КОМПАС и сдают файлы в разных форматах. Учитель открывает 30 работ по одной, чтобы понять, сделан ли «стул» или сдана заготовка. Ученик получает отзыв через неделю, когда уже забыл задание. Кроме того, файлы, скачанные из интернета, внешне не отличаются от собственных.

Идея. Сервис, куда ученик отправляет файл. Сервис конвертирует его, классифицирует и сразу отвечает: «распознано как chair с уверенностью 71 % — задание засчитано» или «похоже на table (60 %), спинка не читается — доработай» (рис. 6). Учитель получает сводную таблицу по классу. Сравнение эмбедингов работ между собой и с моделями из открытых библиотек помечает подозрительно одинаковые файлы.

Цель. Мгновенная обратная связь ученику и сокращение рутинной проверки у учителя; проект применим в собственной школе прямо на следующем уроке.

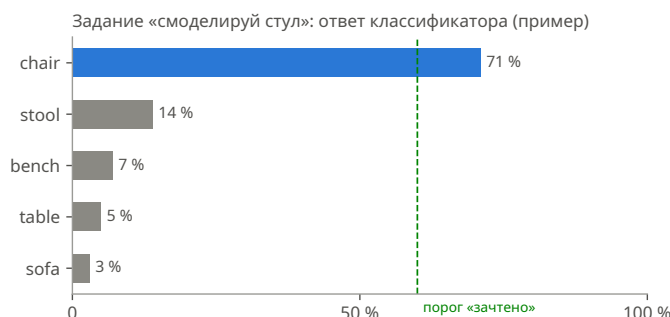


Рис. 6: Ответ классификатора превращается в отзыв: класс задания совпал с верхним предсказанием и вероятность выше порога — «зачтено»; иначе бот показывает, на что модель похожа больше, и предлагает доработать.

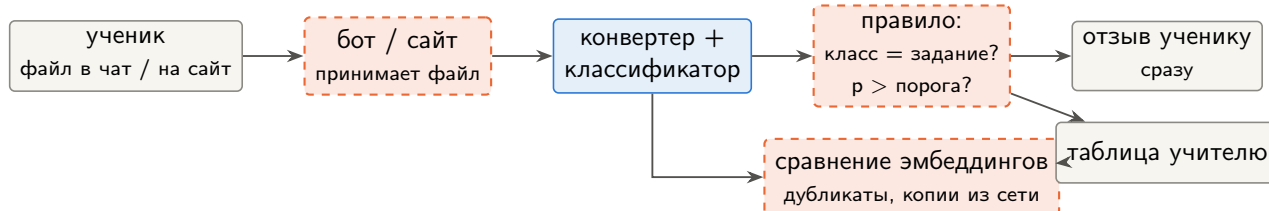


Рис. 7: Схема сервиса проверки. Зелёная (готовая) часть не меняется; вокруг неё — приём файлов, правило зачёта и отчёт.

Этапы работы

1. **Функция проверки** (check.py). Вход: файл и ожидаемый класс; выход: словарь с верхними тремя классами, вероятностями и вердиктом. Порог подобрать на тестовой выборке ModelNet (например, так, чтобы 95 % верных моделей проходили).
2. **Интерфейс.** Самый быстрый путь — Telegram-бот на aiogram: команда /task chair, затем отправка файла. Альтернатива — страница на Streamlit с загрузкой файла и показом 3D-превью.

3. **Список заданий.** JSON с заданиями: класс, срок, порог, подсказка для случая неудачи («у стула должны быть спинка и четыре ножки»).
4. **Отчёт учителю.** CSV/Google-таблица: ученик, файл, класс, уверенность, вердикт, время сдачи.
5. **Антиплагиат** (усложнение). Хранить эмбединг каждой сданной работы; при новой сдаче искать ближайшие; близость выше порога — пометка «проверить вручную». Показать, что сеть отличает скачанный стул от самодельного, — отдельный интересный эксперимент.
6. **Пилот.** Провести одно реальное задание в своём классе, собрать 20–30 файлов, посчитать долю верных вердиктов и опросить одноклассников, полезен ли мгновенный отзыв.

Идея 4. Инвентаризация мебели по 3D-модели помещения

Сложность: ★★★ **Новое обучение:** не требуется **Тип результата:** скрипт-анализатор + отчёт с подписанной картинкой

Проблема. Дизайнеры интерьеров, архитекторы и разработчики игр работают с целыми сценами: комната в одном файле .obj/.glb, в котором тысячи объектов без осмысленных имён (Mesh_0417). Чтобы получить спецификацию — сколько стульев, столов, светильников — сцену приходится просматривать вручную. Та же задача есть у школы: 3D-план кабинета из программы планировки (Planner 5D, SketchUp) и вопрос «сколько чего стоит в кабинете».

Идея. Скрипт разбивает сцену на связанные компоненты (отдельные объекты), нормирует каждый по размеру, классифицирует и выдаёт таблицу-спецификацию: класс, количество, габариты, положение. Результат — ведомость в CSV и изображение сцены с подписанными объектами (рис. 8). Классы ModelNet40 покрывают мебель почти полностью: chair, table, desk, sofa, bed, bookshelf, lamp, night_stand, dresser, wardrobe, tv_stand, piano, plant, monitor.

Цель. Автоматическое составление ведомости по сцене вместо ручного пересчёта; побочно — проверка, что в модели кабинета есть всё, что требуется по нормативу.

Вход: одна сцена в .obj/.glb (сотни тысяч граней)

Выход: разбиение на части, класс и счётчик для каждой

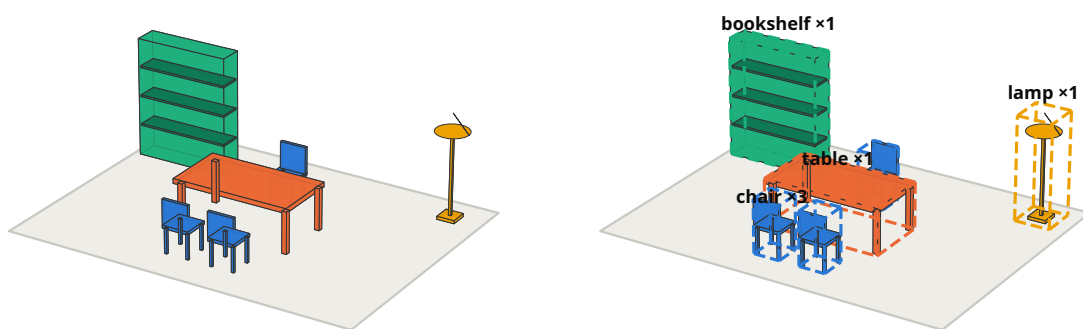


Рис. 8: Сцена делится на отдельные объекты, каждый классифицируется независимо; результат — подписанная сцена и таблица «класс — количество».

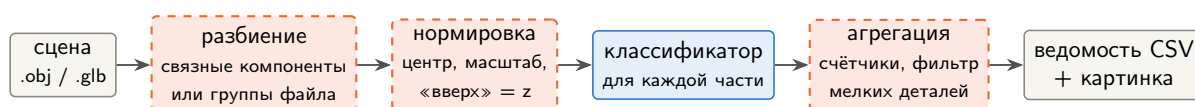


Рис. 9: Конвейер инвентаризации. Главная новая часть — разбиение сцены и приведение каждой части к тому виду, на котором обучалась сеть.

Этапы работы

1. **Разбиение сцены** (`split_scene.py`). Загрузить сцену через `trimesh`; если в файле есть группы/объекты (o/g в OBJ, узлы в GLB) — использовать их; иначе — `mesh.split()` по связности. Отбросить мелочь (объём меньше 0,1 % сцены) и пол/стены (самые большие плоские части).
2. **Нормировка**. ModelNet-модели центрированы, вписаны в единичную сферу и ориентированы осью «вверх». Каждую часть нужно привести к тому же виду, иначе сеть ошибётся. Это самый важный технический шаг — проверить его на 10–20 вырезанных из ModelNet объектах.
3. **Классификация и агрегация**. Прогнать части через сеть, свести в таблицу: класс, число, средние габариты, координаты. Части с низкой уверенностью — в графу «не распознано».
4. **Отчёт**. Изображение сцены с рамками и подписями (`matplotlib` или рендер в Blender скриптом), CSV-ведомость, при желании — HTML-отчёт.
5. **Тесты**. Сделать 3–5 сцен самостоятельно (расставить в Blender модели из ModelNet в комнату) — для них известен правильный ответ, можно посчитать точность подсчёта. Затем попробовать реальную сцену из Sketchfab или свой кабинет в Planner 5D.
6. **Усложнение**. Мебель из реальных сцен часто слита с полом или состоит из нескольких несвязных частей (стол = столешница + 4 ножки). Добавить объединение частей по близости габаритных ящиков — хорошая тема для обсуждения ограничений на защите.

Идея 5. Определитель крепежа и деталей: собственный процедурный датасет

Сложность: ★ ★ ★ ★ **Новое обучение:** да, ModelNet10 + 6–8 новых классов **Тип результата:** новая модель + утилита-определитель + генератор датасета

Проблема. В школьной мастерской, у ремонтника и у любителя 3D-печати есть библиотеки деталей: шестерни, гайки, шайбы, уголки, втулки, болты — сотни файлов из наборов Thingiverse, McMaster-Carr, Fusion 360. ModelNet о таких объектах ничего не знает и назовёт шестерню «чашкой». При этом задача практическая: найти в библиотеке «все шестерни» или понять, что за деталь прислали в файле для печати запчасти.

Идея. Обучить сеть на ModelNet10 (10 бытовых классов) плюс 6–8 классов деталей, но датасет для новых классов не собирать вручную, а *генерировать программно*: каждая деталь описывается несколькими параметрами (число зубьев, радиус, толщина, диаметр отверстия), и скрипт создаёт сотни случайных экземпляров (рис. 10). Это сразу даёт много обучающих данных, идеальную разметку и интересный вопрос для исследования: узнаёт ли сеть, обученная на синтетических деталях, реальные модели из интернета?

Цель. Классификатор, которого нет в открытом доступе (бытовые объекты + технические детали), и воспроизводимый способ расширять его новыми классами за час работы.

Процедурный датасет: класс задан параметрами, по три случайных экземпляра на класс

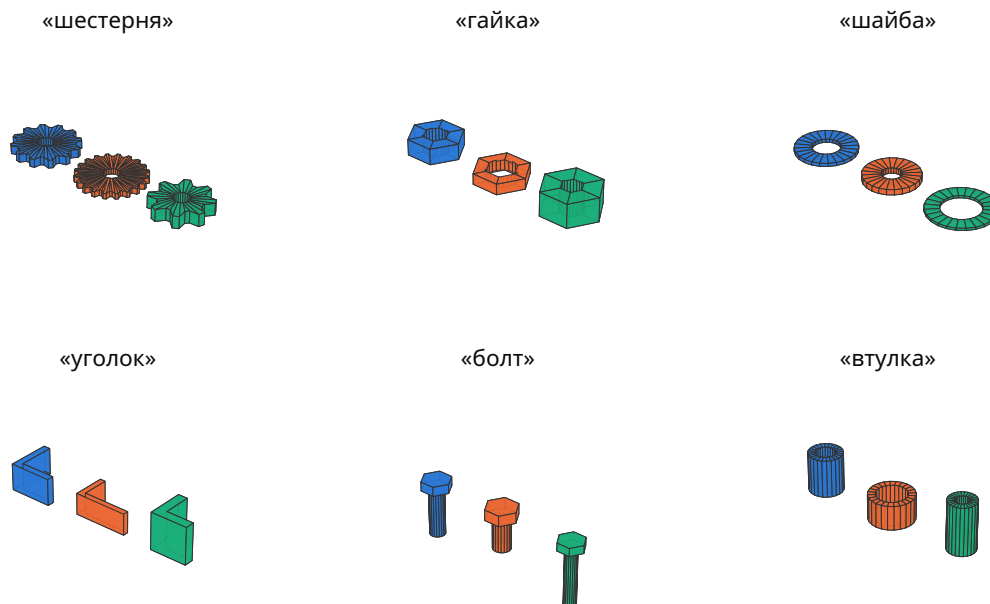


Рис. 10: Процедурная генерация: класс задаётся формулой формы и диапазонами параметров; каждый экземпляр — случайный набор параметров. Генерируются через OpenSCAD (скриптом) или `trimesh.creation` на Python и сразу сохраняются в OFF.

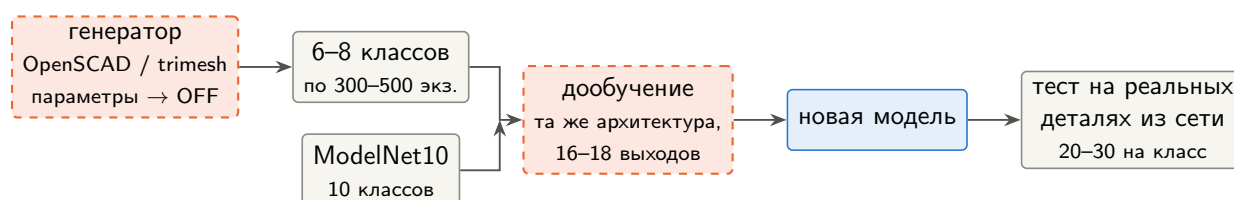


Рис. 11: Схема расширения модели. Ключевой эксперимент — разрыв между точностью на синтетике и на реальных моделях (domain gap) и как его сократить аугментациями.

Этапы работы

1. **Выбор классов.** 6–8 деталей с различной формой: шестерня, гайка, шайба, болт, уголок/кронштейн, втулка, пружина, петля. Для каждого — 4–6 параметров и диапазоны (например, шестерня: зубья 8–40, модуль, толщина, отверстие).
2. **Генератор** (`gen_parts.py`). Для каждого класса — функция, строящая сетку по параметрам; случайный поворот и лёгкий шум вершин как аугментация. Вывод в OFF в той же нормировке, что и ModelNet (центр, единичная сфера).
3. **Сборка датасета.** Объединить с ModelNet10, разделить на train/test с теми же пропорциями (примерно 80/20), сохранить список файлов.
4. **Обучение.** Использовать уже написанный код обучения, изменив число выходов; стартовать с весов текущей модели (transfer learning) — сходится быстрее.
5. **Реальная проверка.** Скачать по 20–30 реальных моделей каждого класса (Thingiverse, GrabCAD) и измерить точность отдельно на синтетике и на реальных файлах. Если разрыв большой — усилить аугментации (шум, случайное масштабирование по осям, обрезка) и повторить: это готовое небольшое исследование.
6. **Утилита-определитель.** Консольная команда `identify.py деталь.stl` → класс, а также пакетный режим для библиотеки (используя код идеи 1).

Почему это сильный проект. Здесь есть всё, что ценят на конференциях: собственные данные, воспроизводимый метод их получения, контрольный эксперимент (синтетика vs. реальность), измеримый результат и практическое применение. Риск — время: генератор и обучение занимают больше, чем кажется; поэтому сначала стоит сделать 3 класса, убедиться, что всё работает, и лишь потом расширять.

Сравнение идей и рекомендация

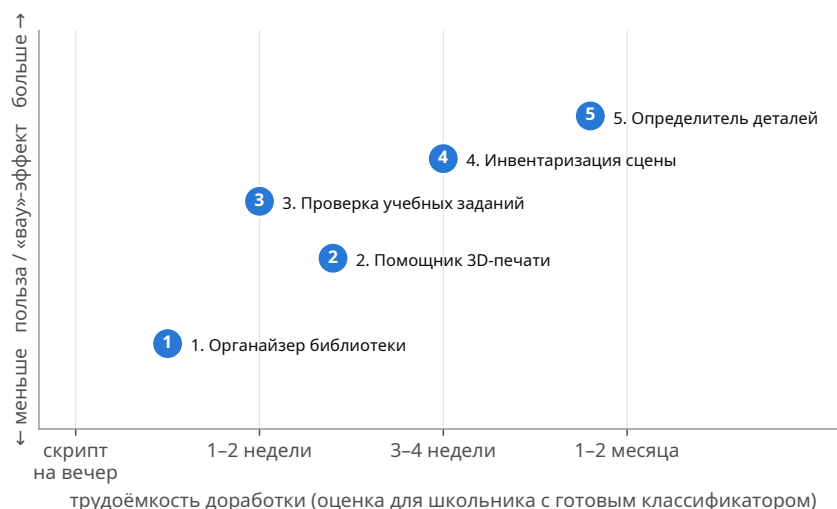


Рис. 12: Субъективная оценка трудоёмкости и отдачи. Идеи 1–3 можно довести до рабочего состояния быстро; идеи 4–5 требуют больше времени, но дают более заметный результат.

	Что добавляется	Новые данные	Главный риск	Кому нужно
1. Органайзер	обход папки, каталог HTML, поиск по эмбедингу	нет	объекты вне 40 классов	кружок 3D-печати, любой пользователь Thingiverse
2. Помощник печати	таблица правил, анализ нависаний, экспорт профиля	по желанию 4–6 классов	класс лишь дополняет геометрию; проверять печатью	начинающие в 3D-печати
3. Автопроверка заданий	бот/сайт, правило зачёта, отчёт, антиплагиат	нет	порог зачёта, работы «не по ModelNet»	учитель и ученики своей школы
4. Инвентаризация сцены	разбиение сцены, нормировка, агрегация, отчёт	нет (тестовые сцены)	сегментация реальных сцен	дизайнеры, архитекторы, школа
5. Определитель деталей	генератор датасета, дообучение, утилита	да, процедурные 6–8 классов	разрыв синтетика/реальность, время	мастерская, ремонт, инженерные библиотеки

Как выбирать

- Если защита через месяц — идея 3 (автопроверка) или 1 (органайзер): результат гарантирован, есть реальные пользователи рядом, можно провести настоящий пилот в классе и привести живые цифры.

- Если есть 2–3 месяца и хочется исследовательской части — идея 5: собственный датасет, эксперимент «синтетика против реальности», а утилита из идеи 1 становится её прикладной частью.
- Если есть 3D-принтер и хочется физического результата — идея 2: сравнение напечатанных объектов «с подсказкой» и «без» — очень наглядный слайд.
- Идеи **комбинируются**: 1+3 (эмбединг общий), 1+5 (органайзер для деталей), 2+5 (профили печати для деталей). Разумная стратегия — сделать идею 1 как базу за первую неделю, а затем выбрать одну из остальных как основную.

Общие советы по оформлению проекта

- В любой идее показать **честную метрику**: точность на тесте ModelNet, а затем точность на «своих» данных, которые всегда хуже. Обсуждение того, почему хуже и что с этим делать, ценится больше, чем высокая цифра.
- Заранее определить, что делать с объектами **вне известных классов**: порог уверенности и класс «не знаю». Это отличает работающий инструмент от демонстрации.
- Хранить код в git, а результаты экспериментов — в CSV: на защите легко показать таблицу, а не пересчитывать.
- Полезные библиотеки: `trimesh` (загрузка, `split`, нормировка, экспорт OFF), `numpy`, `matplotlib/pyrender` (миниатюры), `aiogram/streamlit` (интерфейс), OpenSCAD или `trimesh.creation` (генерация деталей).